FUNDAÇÃO GETULIO VARGAS

ESCOLA DE MATEMÁTICA APLICADA

Sinais Latentes Precoces de Corretude em Geração de Código

Avaliação intratarefa, custo de inferência e transferência de sinais precoces

João Felipe Vilas Boas

Orientador: Prof. Dário Oliveira

Manuscrito de pesquisa — avaliação empírica do desenho fixo

Rio de Janeiro
Outubro de 2026

Resumo

Este estudo avalia sinais latentes de corretude e interrupção precoce na geração de código pelo Qwen2.5-Coder-1.5B-Instruct. Um piloto de 960 trajetórias escolhe o regime de amostragem por regra fixa. O banco principal contém 2.640 tentativas em 132 tarefas HumanEval+, avaliadas pelo EvalPlus. A validação aninhada por tarefa separa treinamento, seleção e teste. No endpoint primário de 32 tokens, 2158 trajetórias permanecem ativas e 92 tarefas contêm ambos os resultados. A diferença de WT-AUC entre H+B e B foi −0,047, IC 95% [−0,099; 0,006]; a diferença de log loss foi 0,038, IC 95% [0,007; 0,071]. O resultado não satisfaz o critério de ganho incremental definido no protocolo. Políticas são comparadas com as mesmas sementes e orçamentos contra parada por verificador e por observáveis. Os critérios conjuntos para afirmar benefício operacional não foram satisfeitos. A transferência aplica um sistema congelado a 640 tentativas MBPP+, sem ajuste nos dados alvo. A execução respeita uma GPU de 6 GB. Os intervalos são condicionais aos modelos OOF ajustados. O histórico de exposição a tarefas é divulgado e não se reivindica pré-registro independente nem aprovação do orientador por inferência a partir da execução.

Palavras-chave: geração de código; probing; corretude funcional; validação agrupada; inferência adaptativa.

Abstract

We evaluate hidden correctness signals and early termination with Qwen2.5-Coder-1.5B-Instruct. A 960-trajectory pilot selects sampling by a fixed rule. The main bank comprises 2,640 attempts across 132 HumanEval+ tasks with official EvalPlus labels and task-grouped nested validation. At the primary 32-token endpoint, 2158 trajectories remain active and 92 tasks contain both outcomes. The H+B versus B within-task AUC difference is −0.047 with a 95% interval [−0.099, 0.006]. The log--loss difference is 0.038 [0.007, 0.071]. The predeclared incremental-gain criterion is not met. Paired policies are compared against verifier stopping and observable-feature aborts. Frozen transfer uses 640 MBPP+ attempts without target fitting. Intervals are conditional on fitted out-of-fold scorers; prior task exposure is disclosed. No independent preregistration or advisor approval is claimed.

Sumário

1 Introdução 1
1.1 Contexto e motivação 1
1.2 Problema científico 1
1.3 Objetivos e sequência de evidências 1
1.4 Delimitação e estado da entrega 2
2 Trabalhos relacionados e posicionamento 3
2.1 Amostragem repetida e verificação funcional 3
2.2 Representações internas e estimação de corretude 3
2.3 Dificuldade e interrupção adaptativa 3
2.4 Contribuição pretendida e limites da reivindicação 3
2.5 Matriz de posicionamento 4
3 Fundamentação matemática e computacional 5
3.1 Formulação probabilística e notação 5
3.2 Transformers, espaço latente e extração causal 6
3.3 Dificuldade da tarefa e sinal específico da trajetória 7
3.4 Amostragem repetida, seleção e custo 9
4 Metodologia 12
4.1 Desenho experimental e construção dos dados 12
4.2 Preditores precoces e modelos estatísticos 13
4.3 Validação, inferência estatística e controles 16
4.4 Política de interrupção precoce e replay 19
4.5 Ambiente efetivamente validado 21
4.6 Transferência externa implementada 21
5 Resultados do desenho fixo 22
5.1 Amostragem, elegibilidade e proveniência 22
5.2 Predição e ganho incremental no endpoint primário 22
5.3 Replay pareado e validação em execução real 24
5.4 Transferência congelada para MBPP+ 26
6 Discussão e limitações 28
6.1 Interpretação dos resultados do desenho fixo 28
6.2 Significado dos padrões encontrados 28
6.3 Validade interna 28
6.4 Validade de mensuração e do verificador 28
6.5 Validade externa e contaminação 29
6.6 Precisão estatística e escopo computacional 29
6.7 Trabalhos futuros 29
7 Considerações finais 30
Referências 31
8 Reprodutibilidade e histórico metodológico 33
8.1 Artefatos e comandos 33
8.2 Correções da versão 0.2 33
8.3 Integridade e assistência computacional 33

1 Introdução

1.1 Contexto e motivação

Modelos de linguagem especializados em código permitem formular a síntese de programas como geração de sequências condicionada a uma especificação. Uma única execução pode produzir uma solução incorreta mesmo quando o modelo atribui probabilidade positiva a soluções corretas. Por isso, amostrar diversos candidatos amplia a oportunidade de encontrar uma solução que passe em um verificador. A avaliação de Codex introduziu o HumanEval e consolidou o uso de corretude funcional e de pass@k na análise de geração de código (Chen et al., 2021). Estudos de amostragem repetida também demonstram que o orçamento de inferência é uma dimensão relevante para ampliar a cobertura de tarefas solucionadas (Brown et al., 2024).

O benefício da repetição tem um custo. Cada candidato utiliza computação para processar o prompt, gerar tokens e executar testes. Parte desse esforço é gasta em candidatos que, quando completados, falham no verificador. Surge então uma possibilidade: identificar durante a geração trajetórias com baixa probabilidade de sucesso e interrompê-las antes de consumir todo o orçamento previsto. Essa possibilidade só é útil se a identificação ocorrer cedo, se não eliminar sucessos em excesso e se seu custo adicional não anular a economia.

O projeto estuda essa questão em um Small Language Model (SLM), usando o termo para descrever o porte do modelo escolhido, sem pressupor uma fronteira universal entre modelos pequenos e grandes. A infraestrutura validada é uma máquina pessoal com GPU NVIDIA RTX 3050 Laptop de 6 GB. A limitação de recursos motiva um desenho compacto, centrado em um modelo, um benchmark principal e uma verificação externa de robustez.

1.2 Problema científico

Um classificador pode prever corretude porque reconhece que certos problemas são fáceis. Isso não implica que consiga distinguir duas respostas diferentes ao mesmo problema. Essa distinção é decisiva para a interrupção de um candidato: uma avaliação que permanece constante entre todas as tentativas do prompt orienta alocação de orçamento por tarefa, mas não identifica qual tentativa merece continuar.

A pergunta central é:

> **Pergunta de pesquisa**
> As representações internas precoces de um SLM contêm informação que distingue trajetórias corretas e incorretas geradas para a mesma tarefa, além da dificuldade do problema e das estatísticas observáveis escolhidas, e essa informação permite reduzir o custo de inferência com múltiplas tentativas?

A expressão “além dos observáveis” refere-se a uma comparação operacional entre classes de preditores especificadas. Não significa informação adicional em relação ao texto completo do prefixo e ao conhecimento integral do modelo, pois o estado interno é calculado a partir desses elementos. O trabalho tampouco interpreta previsibilidade como consciência, introspecção humana ou conhecimento infalível do futuro.

1.3 Objetivos e sequência de evidências

O objetivo geral é caracterizar o valor preditivo e operacional de estados internos de prefixos em geração verificável de código. Três perguntas organizam a investigação:

Pergunta	Evidência necessária
Q1 — Previsibilidade	Medir em que posições os estados internos predizem o resultado final em tarefas não utilizadas no treinamento.
Q2 — Especificidade	Comparar trajetórias corretas e incorretas do mesmo problema e avaliar o ganho sobre estatísticas observáveis do prefixo.
Q3 — Utilidade	Avaliar se uma política de interrupção reduz custo mantendo a resolução dentro de uma margem previamente declarada.

A contribuição pretendida está no encadeamento entre essas evidências. Uma AUC agregada elevada responde apenas a parte de Q1. Uma classificação melhor dentro da tarefa responde a parte de Q2. A existência de informação estatística útil não implica, por si só, que interromper gerações produza ganho operacional.

1.4 Delimitação e estado da entrega

O núcleo experimental utiliza Qwen2.5-Coder-1.5B-Instruct, HumanEval+ e múltiplas gerações sob uma configuração de amostragem congelada. MBPP+ é a única extensão externa planejada. Activation Steering, efeitos causais de intervenções, transferência entre modelos e extração de características por autoencoders esparsos ficam como trabalhos futuros. Não são condições para completar o TCC.

A avaliação principal usa o desenho fixo v0.2, com piloto separado, banco completo e validação por tarefa. A reanálise histórica permanece exploratória e é apresentada separadamente. As decisões empíricas seguem os critérios definidos antes do banco principal; os capítulos divulgam a exposição histórica e a autorização de execução.

O arquivo `RESEARCH_PROTOCOL.md` registra as decisões experimentais. A versão 0.2 incorpora as correções metodológicas descritas no apêndice e aguarda congelamento científico. Não se deve chamar de pré-registrada uma escolha feita depois de observar os resultados que ela pretende confirmar.

2 Trabalhos relacionados e posicionamento

2.1 Amostragem repetida e verificação funcional

Chen et al. (2021) formulam a avaliação de programas gerados por meio de testes e apresentam um estimador de pass@k baseado em múltiplos candidatos. Brown et al. (2024) estudam o aumento de cobertura obtido com mais amostras e destacam a relevância da seleção e verificação. Esses trabalhos motivam a separação entre a capacidade de gerar ao menos uma solução correta e a capacidade de identificá-la.

A qualidade do verificador também importa. Liu et al. (2023) apresentam EvalPlus e a ampliação dos testes de HumanEval para HumanEval+, mostrando que testes mais completos revelam falhas que os testes originais não identificam. Neste projeto, a variável resposta é passar em uma versão fixada dessa suíte. Isso fornece um critério reproduzível, mas não uma prova de correção para todas as entradas possíveis.

2.2 Representações internas e estimação de corretude

Kadavath et al. (2022) diferenciam a avaliação de uma resposta específica da estimativa de que o modelo saberá responder uma pergunta. Essa distinção é próxima da separação entre nível da tarefa e nível da trajetória adotada aqui. Azaria & Mitchell (2023) estudam classificadores sobre ativações internas para identificar veracidade de afirmações, comparando-os também a medidas baseadas na probabilidade da sequência. A abordagem de Representation Engineering organiza a investigação de fenômenos de alto nível em padrões distribuídos das ativações (Zou et al., 2023).

Em geração de código, Di Cicco (2026) reporta decodificação linear de corretude a partir do estado do último token do prompt em Qwen3-4B-Instruct-2507 sobre tarefas de LiveCodeBench. O resultado demonstra a relevância de estados anteriores à geração para discriminação entre tarefas. Por construção, porém, um estado determinístico do mesmo prompt é compartilhado pelas suas diferentes tentativas. Essa observação motiva uma avaliação adicional entre trajetórias do mesmo problema.

Klassert (2026) investiga estimação de incerteza introspectiva em geração de código, com características estáticas e dinâmicas e avaliações no nível da resposta e da linha. O resumo do trabalho reporta sinal de corretude no nível da resposta, sem ganhos consistentes das estratégias dinâmicas mais elaboradas. Isso reforça a opção por um probe simples como experimento principal; não justifica assumir que uma dinâmica temporal mais complexa será superior.

2.3 Dificuldade e interrupção adaptativa

Zhu et al. (2025) estimam dificuldade percebida pelo modelo a partir de representações iniciais e aplicam essa informação a estratégias de inferência adaptativa. Ruan et al. (2026) estudam interrupção de episódios de agentes a partir de probes em estados internos, com controle de retenção e certificação em dados independentes. Seu cenário, unidade temporal e procedimento de garantia diferem de uma regra empírica ajustada em prefixos de código.

O método ST-BoN (Wang et al., 2025) utiliza estados internos em uma estratégia de truncamento e seleção de candidatos. Esse antecedente impede apresentar o uso de hidden states para reduzir o custo de Best-of-N como novidade isolada. A comparação do presente estudo concentra-se na corretude funcional supervisionada, na discriminação dentro da tarefa e na comparação controlada com observáveis.

2.4 Contribuição pretendida e limites da reivindicação

O trabalho não reivindica ter sido o primeiro a prever corretude com hidden states, o primeiro a estimar dificuldade nem o primeiro a interromper inferência com sinais internos. A contribuição

pretendida é uma avaliação integrada e reproduzível, em um regime delimitado de geração de código, de três aspectos: separação entre tarefa e trajetória; poder incremental sobre baselines observáveis especificados; e utilidade em uma política sequencial de interrupção.

As referências são identificadas por fontes primárias, com versão explícita quando fixada na bibliografia. Resultados resumidos não substituem a leitura integral dos métodos antes da comparação final. Em especial, alegações de ausência de trabalhos idênticos exigem revisão bibliográfica mais ampla; não são inferidas de uma busca limitada.

2.5 Matriz de posicionamento

Antecedente	**Objeto principal**	**Relação com este estudo**
Di Cicco	Corretude da primeira tentativa a partir do prompt	Motiva separar dificuldade e trajetória
Klassert	Incerteza introspectiva em respostas e linhas de código	Motiva o probe simples e limites da generalização
Zhu et al.	Dificuldade percebida e alocação de inferência	Comparação no nível da tarefa
Ruan et al.	Abortos em episódios de agentes	Retenção certificada difere de meta empírica
Wang et al. / ST-BoN	Consistência interna entre amostras e truncamento	Uso de estados internos para economia já possui antecedentes
Este estudo	Trajetórias da mesma tarefa, observáveis e replay pareado	Avaliação delimitada; evidência precoce ainda a produzir

3 Fundamentação matemática e computacional

3.1 Formulação probabilística e notação

3.1.1 Tarefas, candidatos e configuração

Seja $\mathcal{X}$ o espaço de tarefas e seja $x_i \in \mathcal{X}$ o prompt da tarefa i, incluindo especificação, assinatura e template fixado. O modelo tem parâmetros θ congelados. Uma configuração c reúne temperatura, top-p, limite de tokens, regras de parada, precisão e versão do tokenizer. A tentativa j produz a sequência aleatória $Y_{ij} = \left(Y_{ij1}, ..., Y_{ijT_{ij}}\right)$, de comprimento T_{ij}.

Distinguem-se a distribuição original do modelo, p_θ, e a distribuição efetivamente usada pelo decodificador, $q_{\theta,c}$. A lei da sequência amostrada é

$$q_{\theta,c}(y \mid x_i) = \prod_{u=1}^{T} q_{\theta,c}(y_u \mid x_i, y_{<u}). \tag{1}$$

O comprimento é determinado por término natural ou limite computacional. A distribuição da sequência incorpora as regras de parada. Tratar todos os candidatos como tendo comprimento fixo esconderia uma dimensão essencial do custo e da elegibilidade para interrupção.

Símbolo	Significado
i, j	Tarefa e tentativa.
x_i, c, θ	Prompt, configuração de inferência e parâmetros do modelo.
T_{ij}	Número de tokens gerados, incluindo os tokens de término conforme convenção registrada.
S_{ij}	Indicador de aprovação na suíte funcional.
ℓ, t	Camada do Transformer e posição relativa à completion.
$h_{ij\ell t}$	Estado interno após processar o token de completion t.
B_{ijt}, z_{ijt}	Vetor de observáveis e score do probe.
π_i	Probabilidade de sucesso de uma tentativa da tarefa i no regime fixado.
N, q	Número máximo de tentativas e limiar de interrupção.
A_i, C_i	Resolução da tarefa e custo acumulado de uma política.

3.1.2 Corretude operacional

Um verificador V recebe a tarefa e o programa extraído da completion. Sua saída define

$$S_{ij} = V\big(x_i, Y_{ij}\big) \in \{0, 1\}, \quad \pi_i = P\big(S_{ij} = 1 \mid x_i, c\big). \tag{2}$$

O verificador é mantido fixo em versão, testes, ambiente e limites de execução. Falhas de sintaxe, execução, asserção e extração são classificadas como insucesso. Um timeout do programa é uma falha sob o orçamento especificado. Uma interrupção da infraestrutura de avaliação é um erro de medição e deve ser reexecutada ou marcada como dado indisponível, sem virar automaticamente um rótulo negativo.

Truncamento no limite de geração é registrado como atributo do candidato. O programa extraído é avaliado pela mesma regra funcional: um candidato que terminou por limite pode, excepcionalmente, já conter uma solução válida. Caso o protocolo deseje definir sucesso como “passa e não truncou”, deve nomear essa variável separadamente e não confundi-la com corretude funcional. Esta versão adota a aprovação no verificador como rótulo principal.

3.1.3 Informação disponível durante a geração

Seja $\mathcal{F}_{ijt}$ a informação observável até o prefixo de comprimento t: prompt, tokens amostrados, distribuições de decodificação calculadas e estados acessíveis até esse instante. O resultado final é observado apenas depois de completar e testar o programa. O alvo ideal de um preditor precoce é

$$p^*_{ijt} = P\big(S_{ij} = 1 \mid \mathcal{F}_{ijt}\big). \tag{3}$$

Antes do término, o mesmo prefixo pode admitir continuações corretas e incorretas. Portanto, "trajetória com baixa chance de sucesso" não significa "trajetória inevitavelmente errada". Dois candidatos que compartilham exatamente o mesmo prefixo podem receber a mesma previsão e posteriormente obter rótulos diferentes.

Pela propriedade da esperança condicional, a sequência ideal p^*_{ijt} satisfaz, antes ou após absorção no término,

$$E\big[p^*_{ij,t+1} \mid \mathcal{F}_{ijt}\big] = p^*_{ijt}. \tag{4}$$

Essa identidade é uma propriedade do preditor probabilístico ideal. Não implica que o probe ajustado seja uma martingala, que a confiança cresça monotonamente em cada trajetória ou que a AUC estimada aumente a cada checkpoint.

3.2 Transformers, espaço latente e extração causal

3.2.1 Representações e atenção causal

Um Transformer representa tokens por vetores e aplica sucessivos blocos de atenção e transformações não lineares (Vaswani et al., 2017). Em notação esquemática, para uma matriz de estados H, uma cabeça de atenção calcula

$$Q = HW_Q, \quad K = HW_K, \quad U = HW_U, \tag{5}$$

$$\mathrm{Attn}(H) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)U. \tag{6}$$

A máscara causal M impede que uma posição consulte tokens posteriores. Conexões residuais, normalizações e transformações feed-forward produzem estados contextualizados. O espaço latente é o espaço vetorial dessas representações, com dimensão determinada pela arquitetura. Não é um espaço de significados com coordenadas semanticamente nomeadas por definição.

O estado $h_{ij\ell t} \in \mathbb{R}^d$ é uma função do prompt e do prefixo processado. Para um modelo determinístico em modo de avaliação,

$$h_{ij\ell t} = f_{\theta,\ell}\big(x_i, Y_{ij,1:t}\big). \tag{7}$$

A aleatoriedade da representação decorre principalmente da amostragem de tokens; pesos e procedimento de extração permanecem constantes. A investigação de padrões lineares nesses vetores testa acessibilidade do sinal a uma classe de decodificadores simples.

3.2.2 Convenção temporal e alinhamento

O checkpoint $t = 0$ corresponde ao último token do prompt. Para $t > 0$, o estado é extraído após processar o token t da completion, e não o estado que foi usado para amostrar esse token. Em um decodificador autoregressivo, os logits usados para amostrar Y_t dependem do prefixo que termina em Y_{t-1}; o estado de Y_t participa da predição seguinte. Confundir essas posições produz um deslocamento de um token e invalida a interpretação temporal.

Um pipeline que para imediatamente depois de amostrar Y_t pode ainda não ter calculado o seu estado. Será necessário processá-lo, sem amostrar Y_{t+1}, para decidir a interrupção. Esse forward adicional ou antecipado deve ser incluído na latência e no custo de operação. Contar t tokens gerados continua correto, mas não contabiliza sozinho todo o trabalho do Transformer.

3.2.3 Propriedade do estado anterior à amostragem

> **Proposição 1 — Ausência de discriminação antes da divergência**
> Com prompt, template, pesos e execução determinística fixados, o estado do último token do prompt é igual entre tentativas da mesma tarefa. Qualquer score determinístico calculado apenas desse estado também é constante dentro da tarefa.

Demonstração. Para todo j, $h_{ij\ell 0} = f_{\theta,\ell}(x_i)$. Logo, $z_{ij0} = g\big(f_{\theta,\ell}(x_i)\big)$ independe de j. Cada comparação entre uma tentativa correta e uma incorreta da tarefa resulta em empate. Uma AUC que atribui peso $\frac{1}{2}$ aos empates vale $\frac{1}{2}$ dentro dessa tarefa.

Esse resultado não impede que `prompt_last` apresente boa AUC entre tarefas. Ele o torna um baseline de dificuldade e um controle de integridade: diferenças relevantes entre repetições do mesmo prompt precisam ser explicadas por mudanças de entrada, dropout, extração incorreta ou não determinismo numérico. Não devem ser interpretadas como previsão da seed futura.

3.2.4 Teste de ausência de informação futura

Para cada camada e posição selecionadas, compare a extração usando apenas o prefixo e a extração na mesma posição de uma passagem sobre a sequência completa:

$$h_{\ell,t}(x, y_{1:t}) \approx h_{\ell,t}(x, y_{1:T}). \tag{8}$$

A igualdade é esperada matematicamente sob atenção causal. Diferenças numéricas podem decorrer de precisão e kernels. O teste deve fixar tolerância absoluta e relativa, modo de avaliação, backend e convenção de normalização. Uma falha exige diagnóstico antes de usar as ativações. Este é um teste de integridade da causalidade temporal da arquitetura, não uma demonstração de que uma direção latente causa corretude.

3.2.5 Alcance informacional do estado interno

Se $H = f_\theta(X, P)$, em que P é o prefixo completo e o modelo é fixado, então H não acrescenta informação aleatória quando (X, P) já é integralmente conhecido. Em notação de informação mútua,

$$I(S; H \mid X, P) = 0. \tag{9}$$

A utilidade do estado está em representar o prefixo de modo que certas propriedades se tornem acessíveis a um preditor limitado. Assim, um ganho sobre NLL e entropia sustenta a afirmação de ganho sobre essas estatísticas, e não sobre todo método possível baseado no texto. Essa restrição impede transformar um resultado de probing em uma alegação excessiva sobre introspecção.

3.3 Dificuldade da tarefa e sinal específico da trajetória

3.3.1 Heterogeneidade entre problemas

A probabilidade π_i varia com a tarefa e a configuração. Em um banco com n_i tentativas, a proporção $\hat{\pi}_i = \sum_j \frac{S_{ij}}{n_i}$ é uma estimativa descritiva dessa probabilidade. Uma tarefa sem sucessos observados não possui necessariamente probabilidade verdadeira zero; a amostra pode simplesmente não ter encontrado sua região de soluções corretas.

Para descrição do banco, definem-se três categorias mutuamente exclusivas: todas as tentativas corretas, resultados mistos e todas as tentativas incorretas. Esses nomes se referem aos resultados observados. Não autorizam afirmar que o modelo sempre acertará ou nunca resolverá o problema.

O número de tentativas influencia a classificação. Sob amostragem independente e n tentativas, a probabilidade de observar resultados mistos é

$$P(\text{mista} \mid \pi_i, n) = 1 - \pi_i^n - (1 - \pi_i)^n. \tag{10}$$

Assim, a análise entre tarefas mistas representa principalmente problemas com variabilidade detectável nesse orçamento. Não é uma amostra aleatória de todos os problemas de programação.

3.3.2 Decomposição da associação

Para um score Z e o indicador S, a lei da covariância total fornece

$$\mathrm{Cov}(Z, S) = \mathrm{Cov}(E[Z|X], E[S|X]) + E[\mathrm{Cov}(Z, S|X)]. \tag{11}$$

O primeiro termo captura a associação entre tarefas; o segundo, a associação dentro da tarefa. A decomposição é conceitual e não uma decomposição da AUC. Ela mostra por que uma associação agregada pode ser explicada inteiramente pela dificuldade do prompt.

Como exemplo estritamente ilustrativo, considere duas tarefas: uma com 90% de acertos e outra com 10%. Um score constante igual a 0,9 para a primeira e 0,1 para a segunda pode ordenar bem exemplos agregados. Entretanto, não distingue qual tentativa acertará dentro de nenhuma das tarefas. Nenhum desempenho numérico desse exemplo é um resultado do modelo estudado.

3.3.3 Discriminação dentro da tarefa

Sejam $\mathcal{P}_i$ e $\mathcal{N}_i$ os conjuntos de tentativas corretas e incorretas elegíveis da tarefa i. Define-se a AUC intratarefa por

$$\mathrm{AUC}_i = \frac{1}{|\mathcal{P}_i|\,|\mathcal{N}_i|} \sum_{j \in \mathcal{P}_i} \sum_{k \in \mathcal{N}_i} \psi\big(z_{ij} - z_{ik}\big), \tag{12}$$

em que $\psi(a)$ vale 1 se $a > 0$, $\frac{1}{2}$ se $a = 0$ e 0 se $a < 0$. A média por tarefas mistas é

$$\text{WT-AUC} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \mathrm{AUC}_i. \tag{13}$$

Cada tarefa informativa recebe o mesmo peso. Uma média por todos os pares de candidatos daria maior peso a tarefas com mais pares positivo-negativo; seria outro estimando. A unidade principal de inferência é a tarefa, mesmo que milhares de pares sejam construídos a partir dela.

Tarefas com apenas uma classe não têm AUC intratarefa definida. Elas continuam nas métricas probabilísticas e operacionais. A exclusão da WT-AUC deve ser acompanhada dos respectivos denominadores, para que um resultado elevado em poucas tarefas não pareça representar todo o benchmark.

3.3.4 Informação incremental e hipótese operacional

Seja B_{ijt} um vetor fixado de características observáveis do prefixo. Compare um modelo que usa apenas B com outro que combina B e a ativação. Os estimandos centrais são

$$\Delta_W = \text{WT-AUC}_{H+B} - \text{WT-AUC}_B, \tag{14}$$

$$\Delta_L = \mathrm{LogLoss}_{H+B} - \mathrm{LogLoss}_B. \tag{15}$$

Valores positivos de Δ_W e negativos de Δ_L indicam ganho, mas os dois critérios avaliam propriedades diferentes: ordenação dentro da tarefa e qualidade de previsão probabilística. Um ganho

robusto sustenta a utilidade incremental da representação para a classe de modelos comparada. Não demonstra independência condicional universal nem que todos os fatores semânticos ou estilísticos foram controlados.

A hipótese nula de ausência de qualquer sinal dentro da tarefa, S independente de Z dado X, difere da hipótese de ausência de informação adicional sobre B, isto é, S independente de Z dado (X, B). Essa distinção orienta a interpretação dos testes estatísticos posteriores.

3.4 Amostragem repetida, seleção e custo

3.4.1 Cobertura e pass@k

Para N candidatos, a cobertura da tarefa é

$$A_i(N) = \mathbf{1}\{\exists j \leq N : S_{ij} = 1\}. \tag{16}$$

Sob tentativas independentes e identicamente distribuídas condicionalmente à tarefa,

$$P(A_i(N) = 1 | x_i) = 1 - (1 - \pi_i)^N. \tag{17}$$

O ganho de passar de $N-1$ para N tentativas é $\pi_i(1-\pi_i)^{N-1}$, que decresce quando $0 < \pi_i < 1$. Essa expressão não deve ser aplicada após substituir π_i por uma taxa global sem considerar heterogeneidade: em geral, a média das coberturas por tarefa difere da cobertura calculada com a média das probabilidades.

Com n candidatos e c_i aprovados, o estimador usual de pass@k é (Chen et al., 2021)

$$\widehat{\text{pass@k}}_i = 1 - \frac{\binom{n-c_i}{k}}{\binom{n}{k}}, \quad k \leq n. \tag{18}$$

Esse estimador descreve a probabilidade de incluir um sucesso em um subconjunto de tamanho k. Já a resolução observada dos primeiros k candidatos é $\mathbf{1}\left\{\sum_{j=1}^{k} S_{ij} > 0\right\}$. Ambos são úteis, mas não são números intercambiáveis. O replay operacional preserva uma ordem de tentativas e utiliza o segundo objeto.

3.4.2 Terminologia: Best-of-N e verificador

"Best-of-N" frequentemente descreve gerar candidatos e escolher o melhor segundo um score. Neste documento, o baseline operacional é mais precisamente **amostragem repetida com verificador e parada no primeiro sucesso**. Ele é denominado BoN com verificador por continuidade com o projeto, com essa definição explícita.

A suíte completa de testes é acessada pela política apenas quando um candidato termina. Esse acesso representa um cenário de verificação forte: o programa não recebe os testes ocultos no prompt, mas o controlador pode utilizá-los para decidir a parada. Isso difere da seleção em produção, na qual os testes disponíveis podem ser incompletos. As conclusões operacionais devem ser limitadas a esse cenário, ou acompanhadas futuramente de uma separação entre testes de seleção e de avaliação final.

3.4.3 Custo até sucesso ou esgotamento

Defina $K_i = \min\{j : S_{ij} = 1\}$, adotando $K_i = \infty$ quando não há sucesso no banco. O custo do baseline até o primeiro sucesso ou limite N é

$$C_i^V(N) = \sum_{j=1}^{N} T_{ij} \mathbf{1}\{K_i \geq j\}. \tag{19}$$

Tarefas não resolvidas contribuem com o custo de todas as tentativas iniciadas. Excluir essas tarefas do custo médio favoreceria políticas que simplesmente desistem de problemas difíceis.

A métrica primária de custo é tokens de completion por tarefa, $\overline{C} = \sum_i \frac{C_i}{m}$. Uma métrica secundária é o custo agregado por tarefa resolvida,

$$C_{\text{por solução}} = \frac{\sum_i C_i}{\sum_i A_i}, \tag{20}$$

que fica indefinido se nenhuma tarefa for resolvida. Ela inclui no numerador o custo das tarefas sem sucesso. Não é a média do custo apenas entre tarefas que tiveram sucesso.

3.4.4 Retenção de candidatos e efeito sobre resolução

Considere uma política que mantém um candidato com indicador R_{ij}, com $R = 1$ para candidatos que terminaram naturalmente antes da decisão. O sucesso útil é $S_{ij}R_{ij}$. Defina $r_i = P(R = 1|S = 1, x_i)$ e $a_i = P(R = 1|x_i)$.

Sob tentativas independentes e a mesma regra por tentativa,

$$P(SR = 1|x_i) = \pi_i r_i, \tag{21}$$

$$P(A_i^\pi(N) = 1|x_i) = 1 - (1 - \pi_i r_i)^N. \tag{22}$$

A precisão entre candidatos mantidos é $P(S = 1|R = 1, x_i) = \pi_i \frac{r_i}{a_i}$, quando $a_i > 0$. Essa precisão pode crescer mesmo quando a cobertura com orçamento fixo diminui. Portanto, "maior fração de acertos entre os mantidos" não basta como evidência de melhoria do sistema.

> **Proposição 2 — Limite da interrupção no mesmo banco**
> Quando a política usa exatamente os mesmos N candidatos potenciais e só pode descartá-los, seu conjunto de sucessos é subconjunto do conjunto de sucessos do baseline. Logo, $A_i^\pi(N) \leq A_i^V(N)$ para cada tarefa.

Demonstração. Se um sucesso é encontrado pela política, o candidato correspondente está no banco original e também passa no verificador do baseline. A política não cria novos candidatos além do limite e não altera o conteúdo dos mantidos. Assim, não pode resolver uma tarefa que nenhum dos N candidatos originais resolvia.

O ganho esperado nesse regime é reduzir custo com perda pequena ou nula de resolução. Para superar cobertura em um mesmo orçamento de tokens, seria necessário reinvestir a economia em novas tentativas, com um banco maior e uma comparação própria. Essa extensão não será confundida com o experimento principal.

3.4.5 Condição econômica para interromper

Uma política pode economizar tokens em um candidato e aumentar o custo total da tarefa ao eliminar um sucesso precoce, forçando outras tentativas. Por isso, o custo do replay não é necessariamente monotônico no limiar de interrupção.

Sob tentativas iid por tarefa, seja D o custo de uma tentativa sob a política e $s = \pi_i r_i$ sua probabilidade de sucesso útil. Como iniciar uma tentativa depende dos resultados anteriores, independentes da tentativa atual,

$$E[C_i^\pi(N)|x_i] = E[D|x_i] \sum_{j=0}^{N-1} (1 - s)^j. \tag{23}$$

Para $s > 0$, a soma vale $\frac{1-(1-s)^N}{s}$; para $s = 0$, vale N. Essa expressão mostra que a redução do custo por tentativa deve compensar a possível redução na probabilidade de sucesso por tentativa. A análise empírica usa os caminhos efetivos do banco e não depende de estimar π_i para substituir nessa fórmula.

3.4.6 Tokens, latência e fronteira custo–resolução

Tokens gerados são uma medida auditável de trabalho de decodificação, mas não equivalem a FLOPs, energia ou tempo. Prefill, comprimento do contexto, cache, extração de estados, cálculo de entropia, transferência GPU–CPU, inferência do probe e verificação têm custos próprios. Os resultados devem distinguir economia de tokens e aceleração medida em execução.

Uma política domina outra no plano descritivo se tem resolução ao menos igual e custo não maior, com uma desigualdade estrita. Na presença de incerteza amostral, a comparação deve reportar diferenças pareadas e intervalos. A fronteira apresentada será a das políticas e limiares previamente definidos; ela não será reinterpretada como uma fronteira ótima global de todos os algoritmos possíveis.

4 Metodologia

A implementação revisada segue o protocolo v0.2, de 12 de outubro de 2026. As regras a seguir definem o experimento principal. Sua implementação e validação técnica não significam que as avaliações confirmatórias tenham sido executadas. A reanálise histórica descrita no capítulo de resultados usa um desenho retrospectivo próprio e não altera os endpoints prospectivos.

4.1 Desenho experimental e construção dos dados

4.1.1 Modelo e ambiente

O modelo principal planejado é Qwen2.5-Coder-1.5B-Instruct, integrante de uma família especializada em código (Hui et al., 2024). Deve-se registrar o identificador completo, revisão imutável dos pesos, tokenizer, template de chat, precisão e backend. A escolha da variante Instruct é específica; não deve ser substituída silenciosamente pela variante base.

A viabilidade na RTX 3050 deve ser medida no piloto. Memória, batch size e eventual quantização dependem da configuração real. Quantização, se necessária, é uma condição experimental que pode modificar as representações; geração e extração devem utilizar a mesma condição. Não se deve gerar candidatos com uma implementação e reconstruir estados de outra sem tratar a diferença como uma análise separada.

4.1.2 Benchmark principal e unidade de generalização

HumanEval+ é o benchmark principal, avaliado pelo EvalPlus. O snapshot usual contém 164 tarefas, cujo número e identificadores serão confirmados no arquivo efetivamente usado. A avaliação de generalização separa tarefas, não apenas tentativas. Problemas muito semelhantes ou duplicados, se identificados antes dos resultados, devem compartilhar o mesmo grupo de particionamento.

O acesso prévio a algumas tarefas para desenvolvimento não desaparece com a geração de novas seeds. Tarefas usadas para decisões de método devem ser registradas como desenvolvimento. O particionamento verifica a generalização a prompts retidos dentro do benchmark, não certifica ausência de contaminação do benchmark no pré-treinamento do modelo.

4.1.3 Piloto e congelamento

O planejamento reserva 32 tarefas, selecionadas deterministicamente pelo hash de `protocol-v0.1-pilot:` concatenado a `task_id`, para desenvolvimento. As demais 132 tarefas constituem o conjunto da avaliação principal. As 32 tarefas não entram nos folds de teste. O histórico de geração prévia em 26 das outras tarefas é declarado; o estudo não presume que essas tarefas fossem desconhecidas do pesquisador. Nenhuma exclusão é feita com base no desfecho observado.

O piloto testa temperaturas 0,6, 0,8 e 1,0, com top-p igual a 0,95, dez tentativas por tarefa e limite inicial de 512 tokens. A finalidade é verificar execução, quantidade de tarefas mistas, taxas de truncamento e custo. AUC de probes não deve ser usada para escolher a temperatura. A regra proposta privilegia o número de tarefas mistas, com taxa agregada de aprovação entre 10% e 90%; empates são resolvidos por taxa mais próxima de 50%, menor comprimento médio e menor temperatura.

O aumento material foi fixado em cinco pontos percentuais acima da menor taxa combinada de sintaxe inválida, extração vazia ou truncamento entre temperaturas. Se nenhuma configuração for elegível, a revisão será registrada antes da análise principal. A coleta parcial não permite escolher uma temperatura.

4.1.4 Banco de candidatos

A configuração definitiva prevê 20 tentativas para cada tarefa confirmatória. Com 132 tarefas, isso corresponde a 2.640 trajetórias potenciais. As ativações e estatísticas são extraídas desse mesmo

banco. Gerar bancos distintos para cada método acrescentaria variação de candidatos à comparação dos métodos.

Cada tentativa tem seed própria, derivada de uma função hash documentada do protocolo, versão do benchmark, tarefa e índice da tentativa, convertida para o intervalo aceito pelo gerador. Deve-se verificar colisões na conversão. A seed é reinicializada antes de cada tentativa, de modo que interromper uma amostra anterior não altere o fluxo aleatório da seguinte.

A independência estatística entre amostras é uma aproximação do desenho de seeds e do decodificador. A reprodutibilidade bit a bit depende também do hardware e das bibliotecas. O banco arquivado, com hashes, é a referência empírica; uma seed isolada não substitui a preservação dos candidatos.

4.1.5 Contrato dos registros

Cada registro contém identificação de tarefa e tentativa, seed, prompt exato, IDs de tokens, completion bruta, código extraído, critério de término, rótulo funcional, classe de erro, custos e referências para as features. O manifesto global contém modelo, tokenizer, benchmark, verificador, versões, configuração, commit e hashes.

Grupo de dados	Campos essenciais
Identidade	`run_id`, `task_id`, `attempt_id`, `seed`, versões do protocolo e do banco.
Geração	Prompt e template, tokens, texto bruto, parâmetros, motivo de parada, comprimento e tempo.
Verificação	Código extraído, aprovação, tipo de falha, versão da suíte e tempo de execução.
Preditores	Camada, checkpoint, ponto de hook, risco ativo, NLL, entropia e caminho/hash das ativações.
Proveniência	Commit, revisões imutáveis, hardware, precisão, dependências e integridade dos arquivos.

Tentativas duplicadas em conteúdo são preservadas se resultam da amostragem prevista. Não se elimina uma solução porque ela parece repetitiva, trivial ou longa. Remoção por corrupção deve conservar o registro da causa e os totais antes e depois. Falhas do programa gerado e falhas da infraestrutura são categorias distintas.

4.1.6 Verificação segura e estável

O código gerado é executado em ambiente isolado, sem credenciais, sem acesso de rede e com limites de tempo, memória e processos. Essas condições protegem a máquina experimental e tornam a medição reproduzível. Reexecuções usam a mesma política de limites. Se o ambiente produz resultados flutuantes, essa instabilidade deve ser diagnosticada antes de usá-la como variável de aprendizado.

O parser de completion para código é determinístico e definido antes da avaliação. Regras para remover cercas Markdown, tratar prefixos e extrair funções não podem variar após a inspeção de exemplos que falharam. Alterar o parser muda o experimento e exige versionar os rótulos.

4.2 Preditores precoces e modelos estatísticos

4.2.1 Checkpoints e conjunto sob risco

São planejados os checkpoints 0, 8, 16, 32 e 64 tokens; a representação final é uma referência tardia. O endpoint confirmatório de especificidade é 32 tokens. As outras posições descrevem a evolução temporal e participam da escolha interna da política operacional.

A ordem da decisão é: primeiro verificar se a geração terminou; se terminou, encaminhar ao verificador; caso contrário, avaliar a regra de interrupção. Assim, para uma decisão depois de t tokens, o conjunto sob risco é

$$\mathcal{R}_t = \{(i,j) : T_{ij} > t\}. \tag{24}$$

O uso da desigualdade estrita evita classificar como abortada uma completion que terminou exatamente no checkpoint. Um estado existente em $T = t$ pode ser usado em uma análise descritiva separada, mas já não corresponde à mesma decisão de continuação. Gerações curtas não são preenchidas com zeros nem recebem uma ativação futura fictícia.

As curvas em diferentes checkpoints condicionam em conjuntos diferentes. Deve-se reportar, em cada ponto, candidatos ativos, tarefas representadas, tarefas mistas e prevalência. Uma análise descritiva adicional sobre o conjunto comum que permanece ativo até 64 tokens ajuda a distinguir mudança de população e mudança de informação; ela representa essa população restrita, e não todo o benchmark.

4.2.2 Camadas e convenção do estado

As quatro profundidades candidatas correspondem aos blocos mais próximos de 25%, 50%, 75% e 100% da profundidade. Os índices são calculados uma vez, registrados e deduplicados se necessário. O manifesto especifica se o vetor corresponde à saída residual do bloco antes ou depois de normalização. O embedding inicial não conta como bloco Transformer.

A captura deve reproduzir o estado disponível na inferência incremental. Salvar todas as ativações de todos os tokens não é requisito: podem-se preservar apenas os vetores selecionados, tokens e metadados, reduzindo o armazenamento. A extração completa offline serve para construir os dados de pesquisa; a política online deverá obter seus estados sem executar o futuro que pretende economizar.

4.2.3 Estatísticas observáveis

Sejam $a_{u,v}$ os logits do token v no passo u. A distribuição bruta é $p_u(v) = \mathrm{softmax}(a_u)_v$. O decodificador aplica temperatura, seleção do núcleo e renormalização para produzir q_u. A probabilidade bruta e a efetivamente amostrada são objetos diferentes.

Para a distribuição de amostragem, definem-se

$$\mathrm{NLL}_u = -\log q_u(Y_u), \quad \mathrm{Ent}_u = -\sum_{v : q_u(v) > 0} q_u(v) \log q_u(v). \tag{25}$$

Até t, as características principais incluem média de NLL, NLL do último token, média de entropia, entropia máxima e média da diferença entre os dois maiores logits. Para esta última, usam-se os logits finitos antes do top-p, após a escala de temperatura, evitando uma margem infinita quando o filtro deixa apenas um token. A base do logaritmo é natural e a unidade é nat.

A logprobabilidade do prefixo é $\sum_{u=1}^{t} \log q_u(Y_u)$. Em um checkpoint fixo, ela é uma transformação linear da NLL média; incluir ambas não cria uma fonte independente de informação. O comprimento final, o tempo total futuro e o resultado dos testes nunca entram em features precoces. O número de tokens t também é constante em uma análise de checkpoint fixo.

Comprimento do prompt e da assinatura são controles de tarefa. Se a assinatura não puder ser extraída com regra confiável, seu indicador de ausência é registrado. Estatísticas equivalentes da distribuição bruta podem ser uma análise de sensibilidade previamente definida, porque entropia após top-p depende do filtro de amostragem.

4.2.4 Baselines e modelo principal

Modelo	Informação utilizada
B0 — Prior	Prevalência do treino, sem rótulos de outras tentativas da tarefa de teste.
B1 — Prompt	Comprimento e atributos observáveis do prompt.
P — Estado inicial	Representação `prompt_last`; baseline de variação entre tarefas.
B — Observáveis	Atributos do prompt e estatísticas de confiança do prefixo.
H — Latente	Estado interno de uma camada e posição escolhidas no treino.
H+B — Combinado	Estado interno concatenado aos mesmos observáveis usados por B.

B, H e H+B utilizam o mesmo tipo de classificador principal: regressão logística com regularização L2. O número de hiperparâmetros e o critério de seleção são controlados. B deve receber seu próprio ajuste de regularização no treino, para que H+B não vença um baseline deliberadamente subajustado.

4.2.5 Probe linear e função objetivo

Para um vetor de features a, o probe define

$$z = b + w^\top a, \quad \hat{p} = \sigma(z) = \frac{1}{1+\exp(-z)}. \tag{26}$$

Se a tarefa i contribui com n_i tentativas elegíveis, cada tentativa recebe peso proporcional a $\frac{1}{n_i}$. Uma forma da função objetivo é

$$J(b,w) = \frac{1}{m_{\mathrm{tr}}} \sum_{i \in \mathcal{I}_{\mathrm{tr}}} \frac{1}{n_i} \sum_{j} \ell\big(S_{ij}, \sigma\big(b + w^\top a_{ij}\big)\big) + \frac{\lambda}{2}\|w\|_2^2, \tag{27}$$

com $\ell(s,p) = -s \log p - (1-s)\log(1-p)$. O intercepto não é penalizado. A relação exata entre λ e o parâmetro `C` depende da normalização usada pela implementação; a configuração preserva a definição da biblioteca, sem assumir que qualquer convenção equivale numericamente a $\lambda = \frac{1}{C}$.

Padronização é ajustada apenas no conjunto de treinamento de cada split. Uma direção por diferença de médias,

$$v = \hat{\mu}_1^{\mathrm{tr}} - \hat{\mu}_0^{\mathrm{tr}}, \tag{28}$$

pode ser um diagnóstico geométrico secundário. A direção e sua orientação são construídas exclusivamente com rótulos de treino. Separação visual em projeções escolhidas depois de olhar todo o banco não constitui evidência confirmatória.

4.2.6 Calibração e interpretação dos scores

Uma transformação logística de calibração, $\hat{p}_{\mathrm{cal}} = \sigma(az + b)$, pode ser ajustada com previsões de desenvolvimento produzidas fora da amostra. Calibração trata a relação entre score e frequência observada de sucesso; é distinta de discriminação (Guo et al., 2017). Uma transformação estritamente crescente preserva a ordenação e a AUC.

A interrupção por limiar necessita principalmente de uma boa ordenação e de um limiar validado. Probabilidades bem calibradas facilitam interpretação e comparações, mas não são condição matemática para obter economia. Será reportado o desempenho probabilístico, sem apresentar calibração como garantia de retenção de corretos em tarefas novas.

4.3 Validação, inferência estatística e controles

4.3.1 Separação por tarefa e avaliação aninhada

As tentativas de um problema compartilham especificação e estrutura de solução. Dividi-las aleatoriamente entre treino e teste permite que o treinamento se beneficie de características do próprio problema avaliado. A separação exigida é

$$\mathcal{I}_{\mathrm{tr}} \cap \mathcal{I}_{\mathrm{te}} = \varnothing, \tag{29}$$

em que os conjuntos contêm identificadores de tarefas. As mesmas tarefas pertencem ao mesmo fold em todos os checkpoints e métodos. Não se refaz o particionamento para obter uma distribuição mais favorável de rótulos depois dos resultados.

Utiliza-se avaliação externa com cinco folds agrupados e validação interna com três folds agrupados dentro de cada treino externo. A camada e `C` são selecionados internamente. O risco de sobreajuste ao critério de seleção motiva a separação entre seleção e avaliação (Cawley & Talbot, 2010). Todos os passos dependentes dos dados, inclusive padronização e construção de direções, são repetidos no treino correto.

O conjunto candidato de regularização é $C \in \{0.001, 0.01, 0.1, 1, 10\}$. No endpoint científico de 32 tokens, a posição não é selecionada. Para cada modelo, o critério interno principal é log loss médio por tarefa. Dentro de uma faixa de um erro-padrão do melhor valor interno, usa-se maior regularização e depois camada mais rasa. O erro-padrão para essa regra é uma heurística de seleção, não um intervalo de confiança do resultado final.

O objeto avaliado pela validação externa é o procedimento que escolhe, treina e aplica um modelo usando apenas tarefas de treino. As camadas selecionadas podem variar entre folds. Não se deve apresentar a melhor camada encontrada posteriormente em todo o banco como se tivesse sido definida antecipadamente.

4.3.2 Predições fora da amostra e calibração interna

Cada candidato confirmatório recebe uma previsão de um modelo que não treinou em sua tarefa. A concatenação dessas previsões permite calcular métricas sobre o conjunto externo completo. As escolhas continuam restritas ao interior de cada treino externo.

Para calibrar e selecionar a política, obtêm-se scores internos fora da amostra. Quando a seleção de hiperparâmetros participa desse processo, ela também precisa ocorrer sem o fold interno cujos scores estão sendo produzidos, ou os scores devem ser tratados apenas como dados de desenvolvimento com viés de seleção. A implementação v0.2 produz scores por cross-fitting da seleção inteira: cada partição pontuada é excluída também da escolha de camada e regularização. O calibrador é ajustado nesses scores; calibrador e limiar continuam sendo ajustes de desenvolvimento avaliados integralmente em tarefas externas.

Calibrador e limiar podem ser ajustados nos dados de desenvolvimento desde que todo o sistema seja avaliado depois em tarefas externas intocadas. O recall obtido nesse conjunto de ajuste é uma meta empírica, sujeita a otimismo por seleção. Para reivindicar uma certificação de recall, seria necessário um conjunto independente de certificação e um procedimento de inferência adequado à dependência entre tentativas; o presente desenho não faz essa reivindicação.

4.3.3 Métricas probabilísticas e de ranking

As métricas principais de probabilidade são log loss e Brier, com peso igual por tarefa:

$$\mathrm{LogLoss} = \frac{1}{m} \sum_{i} \frac{1}{n_i} \sum_{j} \ell\big(S_{ij}, \hat{p}_{ij}\big), \tag{30}$$

$$\text{Brier} = \frac{1}{m}\sum_i \frac{1}{n_i}\sum_j \left(\hat{p}_{ij} - S_{ij}\right)^2. \tag{31}$$

Para estabilidade numérica, probabilidades usadas em logaritmos são limitadas a $[\varepsilon, 1-\varepsilon]$, com ε documentado e idêntico entre métodos. A AUC agregada é reportada como diagnóstico entre candidatos, acompanhada da WT-AUC. O resumo de precisão-revocação utiliza average precision (AP), com implementação declarada; a denominação genérica PR-AUC não deve esconder a diferença entre AP e integração trapezoidal.

Curvas de calibração descrevem a relação entre probabilidade prevista e frequência observada. O número de bins e a regra de particionamento são fixados. ECE é secundário porque depende desses bins e pode ser instável na amostra pequena. A análise não se limita a acurácia com limiar 0,5, que pode ser pouco informativa para a política de alta retenção.

4.3.4 Bootstrap por tarefa e alcance dos intervalos

Para estimar incerteza, reamostram-se tarefas inteiras com reposição, preservando todas as suas tentativas, scores e resultados de políticas. Usam-se os mesmos índices reamostrados para comparar os métodos. Em cada réplica, recomputam-se as métricas e as diferenças. O número planejado é 2.000 réplicas com seed fixada, produzindo intervalos percentílicos de 95%.

Na WT-AUC, reamostrar apenas as tarefas mistas estima a incerteza condicional a esse subconjunto; nas métricas de resolução e custo, reamostra-se todo o conjunto confirmatório. A opção e seu denominador serão explicitados. Réplicas sem classes necessárias não recebem valor zero artificialmente; são contadas como indefinidas.

O bootstrap dos scores externos já calculados trata os ajustes como fixos. Como os modelos dos folds compartilham dados de treino, esses intervalos são aproximações para a variabilidade entre tarefas e não capturam integralmente a instabilidade de retreinar todo o procedimento. Uma análise adicional com repetição agrupada do processo pode avaliar essa componente, mas precisa ser dimensionada antes de olhar os resultados. O texto final deverá dizer se a incerteza reportada é condicional aos modelos ajustados.

4.3.5 Permutações: hipótese nula e validade

Permutações avaliam se o alinhamento entre features e rótulos é compatível com uma hipótese nula de intercambialidade (Ojala & Garriga, 2010). Embaralhar rótulos dentro da tarefa mantém seu número de acertos e destrói o alinhamento entre prefixos e desfechos. Esse é um controle relevante para sinal intratarefa.

Entretanto, tal embaralhamento também destrói a relação entre os observáveis B e o resultado. Por isso, ele não é um teste específico da hipótese “hidden states não acrescentam nada a B”. Aplicar esse procedimento ao delta H+B menos B e chamá-lo de teste de poder incremental seria uma interpretação incorreta. A análise incremental principal usa deltas pareados fora da amostra, intervalos por tarefa e perdas probabilísticas.

Há também um cuidado com os folds. Permutar todos os rótulos depois de concatenar scores OOF e tratar todos esses scores como externos a todos os rótulos ignora que o rótulo de uma tarefa pode ter sido usado para treinar scores de outros folds. Uma alternativa computacionalmente simples é o teste por fold externo:

1. Fixar o modelo e as previsões do fold k, sem ajustes usando seus rótulos.
2. Permutar os rótulos apenas dentro de cada tarefa ativa desse fold.
3. Calcular a WT-AUC de H+B em cada permutação, usando 10.000 repetições.
4. Obter $p_k = \frac{1+\sum_b \mathbf{1}\{M_b \geq M_{\text{obs}}\}}{10001}$, com M igual à WT-AUC.

5. Combinar conservadoramente por $p_{\text{global}} = \min(1, 5 \min_k p_k)$.

A correção de Bonferroni permanece válida sem independência entre os cinco p-valores, desde que cada teste individual seja válido. Folds sem tarefas mistas recebem $p_k = 1$. Esse teste verifica ausência global de associação intratarefa no sentido definido; a combinação pode ter potência baixa. Uma alternativa é permutar rótulos dentro das tarefas e repetir integralmente treinamento e seleção a cada permutação, com custo computacional maior. A escolha deve ser congelada, sem selecionar a estratégia que produza menor p-valor.

A validade ainda pressupõe que, sob a hipótese nula, os rótulos sejam intercambiáveis entre tentativas elegíveis do mesmo problema. Amostragem com configurações distintas ou tentativas dependentes de feedback do verificador exigiria restringir os blocos de permutação ou redesenhar o teste. Este projeto usa tentativas sem realimentação entre gerações.

4.3.6 Efeitos fixos como análise complementar

Uma regressão com intercepto por tarefa formaliza o controle da informação constante no prompt:

$$\text{logit } P\big(S_{ij} = 1\big) = \alpha_i + \beta_B^\top B_{ijt} + \beta_z z_{ijt}. \tag{32}$$

A estimação logística condicional pode eliminar os interceptos ao condicionar no número de sucessos de cada tarefa. Tarefas sem variação de rótulo não identificam β_z. Colunas constantes dentro da tarefa, como comprimento do prompt, são absorvidas pelo efeito fixo e não precisam ser estimadas separadamente.

Essa regressão é associativa e complementar. Interceptos estimados com os rótulos das tarefas externas não constituem um preditor válido para tarefas novas. Portanto, não se usa sua log loss ajustada como melhoria fora da amostra. A comparação preditiva vem dos modelos B e H+B avaliados externamente. A incerteza de um coeficiente calculado sobre scores aprendidos deve reconhecer que esses scores também foram estimados; um erro-padrão convencional não representa automaticamente toda a cadeia de aprendizado.

4.3.7 Controles negativos e diagnósticos

O controle determinístico de `prompt_last` deve ter WT-AUC igual a 0,5 quando há empates exatos. Uma direção aleatória normalizada avalia a facilidade de obter associação aparente em alta dimensão. Um probe treinado com rótulos permutados dentro das tarefas do treino verifica se a separação depende do alinhamento real entre representação e resultado.

A direção invertida não é um controle independente contra associação: para um score sem outras mudanças, $\text{AUC}(-z) = 1 - \text{AUC}(z)$ com a convenção de empates. Ela serve para verificar sinal e implementação. Também não se permite usar o comprimento final como controle de uma política precoce. Indicadores de formatação já presentes no prefixo podem ser diagnósticos de artefatos; seu ajuste deve respeitar os splits.

Um desempenho não significativo de um controle não demonstra automaticamente a validade do modelo principal. Os controles se somam a uma avaliação explícita de vazamento, amostra, hipótese nula e tamanho de efeito.

4.3.8 Multiplicidade e critérios de evidência

A posição de 32 tokens e a comparação H+B contra B definem o endpoint principal. O resultado positivo de especificidade incremental requer $\Delta_W > 0$, intervalo pareado de 95% inteiramente acima de zero e $\Delta_L < 0$ fora da amostra. O intervalo de Δ_L também é reportado; se ele incluir zero, a evidência probabilística é qualificada como incerta. O teste de permutação apoia a análise de existência de sinal intratarefa, sem funcionar como teste do ganho condicional.

Curvas por posição, comparações de camadas, tipos de erro, direções geométricas e subgrupos são análises secundárias. Se forem submetidas a testes formais múltiplos, a família de comparações e a correção serão declaradas. Não se escolhe retrospectivamente outro checkpoint para substituir um resultado desfavorável em 32 tokens.

Não rejeitar uma hipótese nula não demonstra equivalência. Um intervalo amplo pode conter tanto ausência de efeito quanto um efeito relevante. Com poucas tarefas mistas, a conclusão apropriada pode ser "inconclusivo neste orçamento", e não "o modelo não contém o sinal".

4.4 Política de interrupção precoce e replay

4.4.1 Regra de decisão e casos de término

A política utiliza um único checkpoint t^* por ajuste, evitando a multiplicação de decisões dentro da mesma tentativa. Se o candidato permanece ativo nesse ponto e seu score é inferior ao limiar q, a geração é interrompida. Caso contrário, continua normalmente até terminar. Não há steering nem alteração da distribuição dos tokens nos candidatos mantidos.

O algoritmo por tarefa é:

1. Iniciar a tentativa seguinte com sua seed pré-atribuída.
2. Gerar até o término natural, o checkpoint ou o limite, conforme o que ocorrer primeiro.
3. Se houve término, extrair o programa, verificar e parar a tarefa se houver sucesso.
4. Se a geração segue ativa no checkpoint, calcular os preditores e o score.
5. Interromper se o score for menor que q; caso contrário, completar e verificar.
6. Repetir até um sucesso verificado ou até atingir N tentativas iniciadas.

Uma tentativa abortada consome orçamento de tentativa. Um candidato que termina exatamente em t^* vai ao verificador antes da decisão de aborto. A regra com q abaixo de todos os scores representa "não interromper" e deve sempre estar incluída na seleção.

4.4.2 Seleção do checkpoint e do limiar

Dentro do treino externo, avaliam-se $t \in \{8, 16, 32, 64\}$ e limiares derivados exclusivamente dos scores de desenvolvimento. O modelo de cada posição é ajustado sem usar o teste externo. A métrica de retenção relevante para o gate é

$$\hat{r}_{\text{ativo}}(t, q) = \frac{\sum_{i,j} S_{ij} \mathbf{1}\{T_{ij} > t\} \mathbf{1}\{\hat{p}_{ijt} \geq q\}}{\sum_{i,j} S_{ij} \mathbf{1}\{T_{ij} > t\}}. \tag{33}$$

Essa taxa evita que muitos sucessos curtos, nunca expostos ao gate, escondam perdas entre candidatos efetivamente avaliados. Reporta-se também a retenção global, contando sucessos que terminaram antes do checkpoint como mantidos. Se não há sucessos ativos suficientes para avaliar uma configuração, ela não é calibrável e a opção conservadora é não interromper.

A meta de desenvolvimento é $\hat{r}_{\text{ativo}} \geq 0.97$. Todos os pares checkpoint-limiar que atendem à meta são avaliados por replay de custo médio por tarefa. Escolhe-se o menor custo; empates numéricos são resolvidos por maior resolução no desenvolvimento, checkpoint mais cedo e menor limiar. Não se escolhe automaticamente o maior limiar admissível, pois abortar sucessos pode aumentar o custo até solução.

Para evitar impor um limiar pior que o baseline, a opção sem aborto participa da mesma seleção. O protocolo registra uma tolerância numérica para empates e uma grade finita de limiares, incluindo os valores nos quais a decisão muda. A política e seu calibrador ficam congelados antes da avaliação externa.

4.4.3 Meta empírica de recall e generalização

A taxa de 97% é uma meta de ajuste. O recall no teste deve ser medido, e pode ser menor. Dependência dentro da tarefa, seleção do melhor limiar e mudança de distribuição impedem atribuir automaticamente uma garantia binomial simples à taxa agregada.

Ruan et al. (2026) distinguem ajuste e certificação independente em seu contexto de agentes. Esta proposta adota apenas uma restrição empírica de seleção e uma avaliação externa; não reproduz a garantia formal daquele método. Além disso, retenção de trajetórias corretas não é igual a preservação da resolução das tarefas. Perder a única solução de um problema tem efeito diferente de perder uma entre várias soluções corretas.

4.4.4 Replay com candidatos pareados

O banco guarda a completion completa de cada tentativa. Para simular aborto, o replay consulta apenas features disponíveis no checkpoint e usa o resultado completo exclusivamente para avaliar o contrafactual de retenção: saber se o candidato descartado passaria. A política não acessa esse rótulo para decidir.

Defina $b_{ij} = \mathbf{1}\{T_{ij} > t^*, \hat{p}_{ijt^*} < q\}$. O custo de uma tentativa iniciada no replay é

$$d_{ij}^{\pi} = b_{ij} t^* + (1 - b_{ij}) T_{ij}. \tag{34}$$

Se K_i^{π} é a primeira tentativa não abortada que passa, então

$$C_i^{\pi}(N) = \sum_{j=1}^{N} d_{ij}^{\pi} \mathbf{1}\{K_i^{\pi} \geq j\}, \tag{35}$$

$$A_i^{\pi}(N) = \mathbf{1}\{K_i^{\pi} \leq N\}. \tag{36}$$

É essencial somar apenas tentativas efetivamente iniciadas. O rótulo de um candidato abortado pode ser usado para calcular false-abort na análise, mas não para parar o replay como se a tarefa tivesse sido resolvida.

A equivalência com a execução online exige que abortar uma tentativa não modifique a identidade das seguintes. Seeds independentes, ordem fixa e configuração determinística fornecem essa propriedade no desenho. Batching e diferenças numéricas podem impedir igualdade bit a bit; por isso, uma execução online pequena deve conferir decisões, contagem de tokens e comportamento de término.

4.4.5 Comparadores operacionais

O experimento utiliza o mesmo banco e a mesma ordem para quatro famílias: geração fixa de N candidatos completos; geração sequencial com verificador; interrupção com observáveis B; e interrupção com H+B. Os orçamentos reportados são 1, 5, 10 e 20, com $N = 20$ como comparação principal.

Cada política adaptativa recebe o mesmo espaço de checkpoints, método de calibração e regra de seleção. Uma ablação secundária pode fixar o mesmo checkpoint para B e H+B, isolando a contribuição do score. A comparação principal entre procedimentos permite que cada um selecione sua configuração dentro do mesmo orçamento de desenvolvimento.

O baseline fixo e o baseline com verificador têm a mesma cobertura potencial no mesmo conjunto de candidatos, mas custos distintos quando há sucessos precoces. O comparador decisivo da economia é o baseline com verificador, porque ele já evita gerar tentativas depois de resolver a tarefa.

4.4.6 Não inferioridade e economia

Definam-se as diferenças médias pareadas

$$\Delta_A = \frac{1}{m}\sum_i (A_i^{\pi} - A_i^{V}), \quad \Delta_C = \frac{1}{m}\sum_i (C_i^{\pi} - C_i^{V}). \tag{37}$$

A margem planejada de não inferioridade é 0,03 em proporção, isto é, 3 pontos percentuais. Exige-se que o limite inferior do intervalo de 95% de Δ_A seja maior que -0.03, e que o limite superior do intervalo de Δ_C seja menor que zero. Usar um intervalo bilateral de 95% para essas condições é uma escolha conservadora em relação a um teste unilateral de 5%.

A economia relativa é $1 - \frac{\overline{C}^{\pi}}{\overline{C}^{V}}$, com intervalo pareado. A margem de resolução não deriva automaticamente do recall de 97%; são critérios distintos. Com aproximadamente 132 tarefas, a perda de uma tarefa equivale a cerca de 0,76 ponto percentual, o que limita a resolução estatística de uma margem estreita. A discussão de potência deve considerar tarefas e discordâncias pareadas, não apenas o número total de trajetórias.

A comparação contra a política observável usa as mesmas métricas. Economizar em relação ao verificador, mas não superar B, sustenta o valor da interrupção naquele regime, sem demonstrar que os estados internos foram necessários para o ganho.

4.4.7 Medição online e amortização

A execução online mede tempo após aquecimento, sincronização apropriada da GPU e condições comparáveis de carga e batch. Reporta geração, extração/score, verificação e total. A política não deve ser creditada por economia de FLOPs ou energia sem medição correspondente.

O custo de pesquisa inclui gerar o banco completo, extrair ativações, treinar e calibrar. Esses custos não pertencem ao custo marginal por tarefa da política já treinada, mas precisam ser informados. Se o custo de preparação for F e a economia média por uso for $s > 0$ na mesma unidade, o ponto de amortização idealizado é $\frac{F}{s}$ utilizações. Sem estimativas compatíveis, não se afirma retorno econômico total.

4.5 Ambiente efetivamente validado

O dispositivo é uma NVIDIA GeForce RTX 3050 6GB Laptop GPU. A geração usa pesos em bfloat16, projeções lineares calculadas em float32, atenção eager e uma tentativa por vez, com TF32 desativado. O armazenamento dos vetores selecionados usa float16. Essa combinação separa precisão dos pesos, computação e armazenamento; descrevê-la apenas como "inferência FP16" seria inexato. As tentativas técnicas rejeitadas não forneceram rótulos ao banco. Versões e hashes estão nos manifestos.

4.6 Transferência externa implementada

O fluxo secundário prepara modelos, calibradores e políticas exclusivamente com HumanEval+, congela o bundle e então permite coletar MBPP+. São planejadas 32 tarefas selecionadas por hash, com 20 tentativas cada. A aplicação não ajusta parâmetros com rótulos do benchmark alvo. Recalibração ou retreino em MBPP+ constituiriam outro experimento. Os resultados da transferência congelada são reportados no capítulo empírico, sem retreino, recalibração ou seleção nos rótulos alvo.

5 Resultados do desenho fixo

5.1 Amostragem, elegibilidade e proveniência

O piloto e o banco principal são separados por tarefa. A coleta mantém todas as 20 tentativas de cada tarefa principal, inclusive após um sucesso, para permitir replay pareado. No endpoint primário há 2158 tentativas elegíveis, 132 tarefas e 92 tarefas mistas. Tarefas homogêneas contribuem para perdas e resolução, mas não identificam AUC intratarefa. A população ativa muda entre checkpoints: as curvas temporais não acompanham uma amostra invariável.

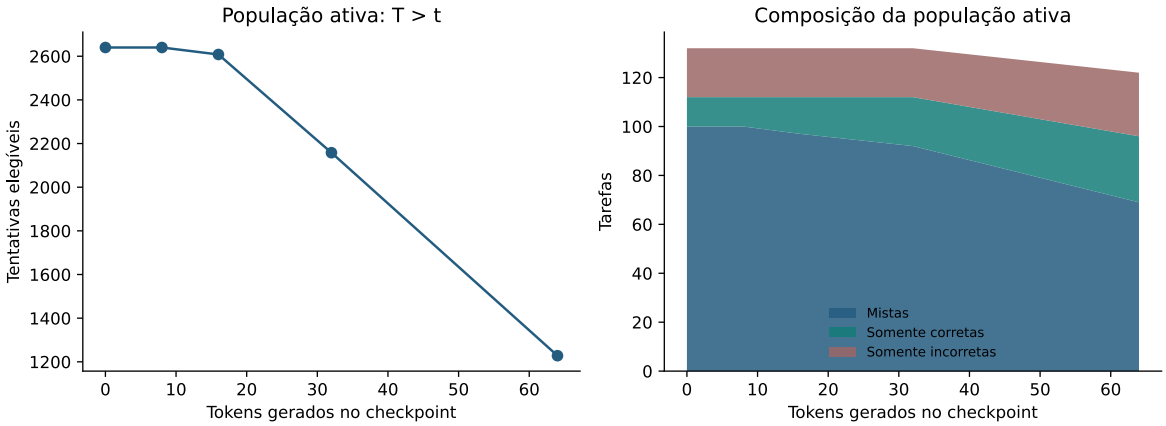


Figura 1: Elegibilidade T > t e composição das tarefas. Checkpoint zero usa todos os prompts.

5.2 Predição e ganho incremental no endpoint primário

Preditor	AUC	WT-AUC	IC 95% WT-AUC	Log loss
B	0,701	0,628	[0,583; 0,675]	0,630
H	0,592	0,568	[0,521; 0,614]	0,687
H+B	0,646	0,581	[0,534; 0,629]	0,668

Tabela 1: Predições externas em 32 tokens; cada tarefa recebe o mesmo peso total.

A diferença H+B menos B em WT-AUC é −0,047, IC 95% [−0,099; 0,006], e em log loss é 0,038, IC 95% [0,007; 0,071]. Esse resultado não satisfaz o critério conjunto de Q2. O intervalo de perda permanece acima de zero: H+B apresenta maior perda que B neste contraste. O teste de permutação por fold avalia associação intratarefa; ele não constitui um teste isolado de informação adicional sobre B. Os controles e as métricas secundárias estão apresentados para diagnóstico, sem substituir o endpoint.

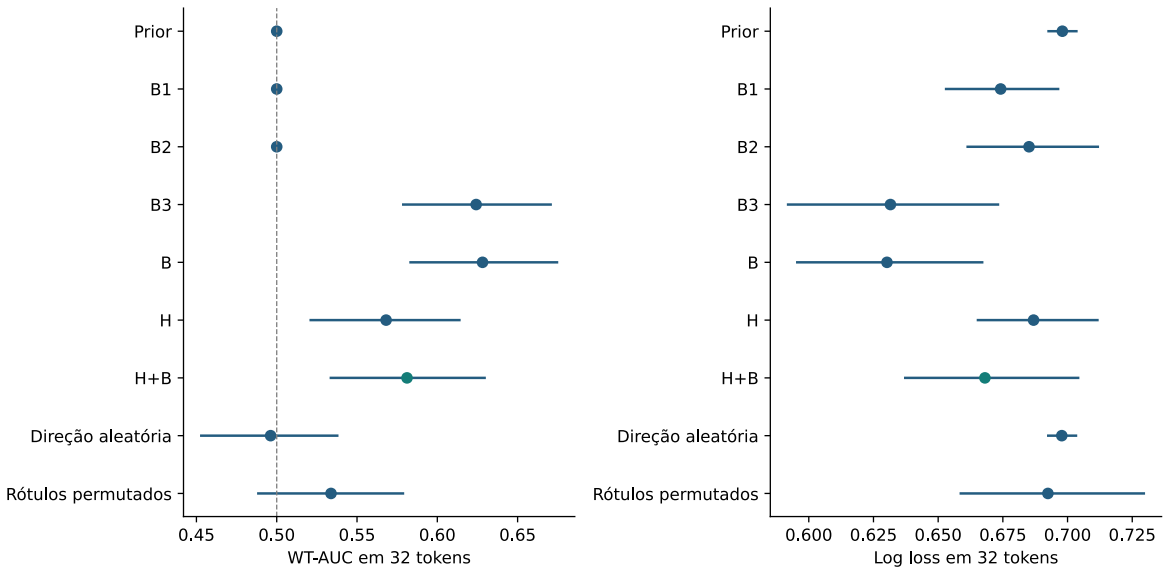


Figura 2: Desempenho primário e controles. Intervalos por bootstrap de tarefas, condicionais aos modelos ajustados.

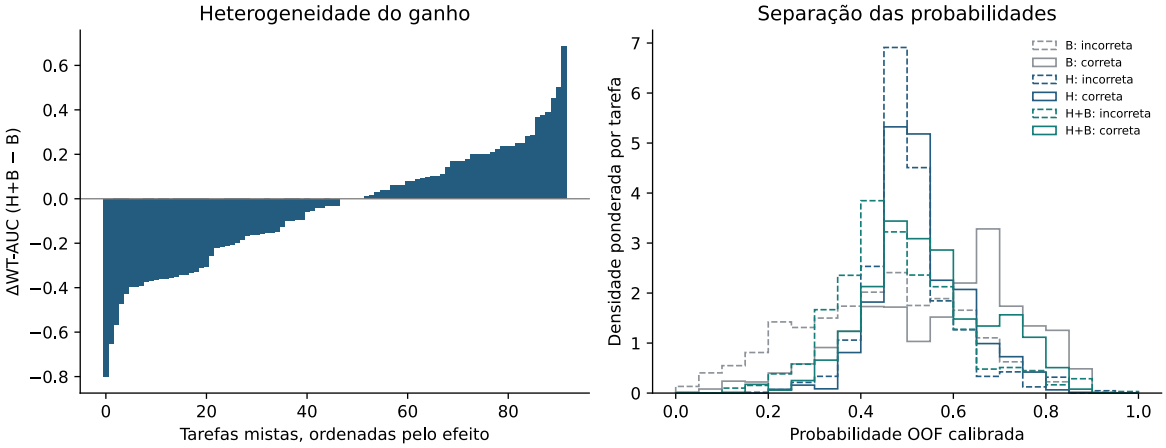


Figura 3: Heterogeneidade do incremento por tarefa mista e distribuição ponderada das probabilidades OOF.

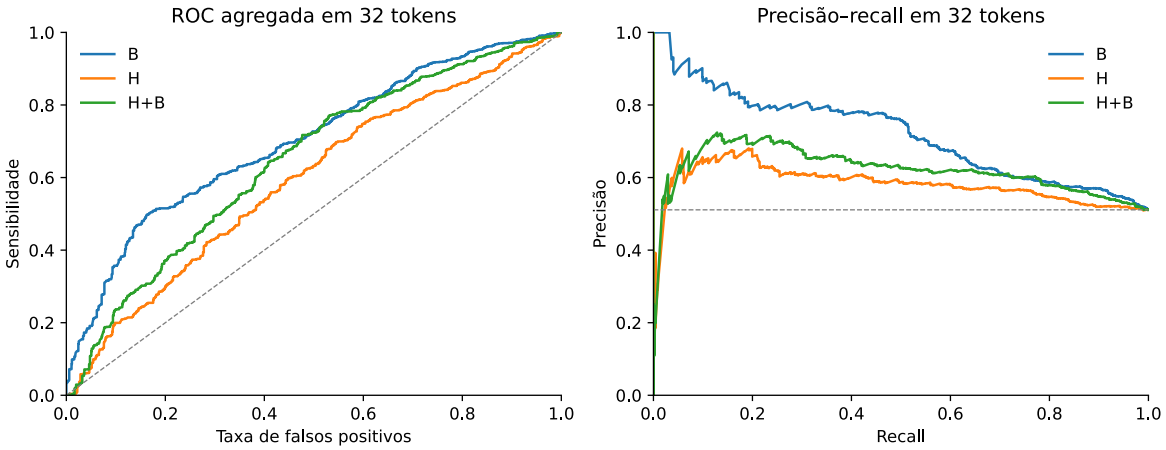


Figura 4: Curvas agregadas ROC e precisão–recall ponderadas por tarefa em 32 tokens.

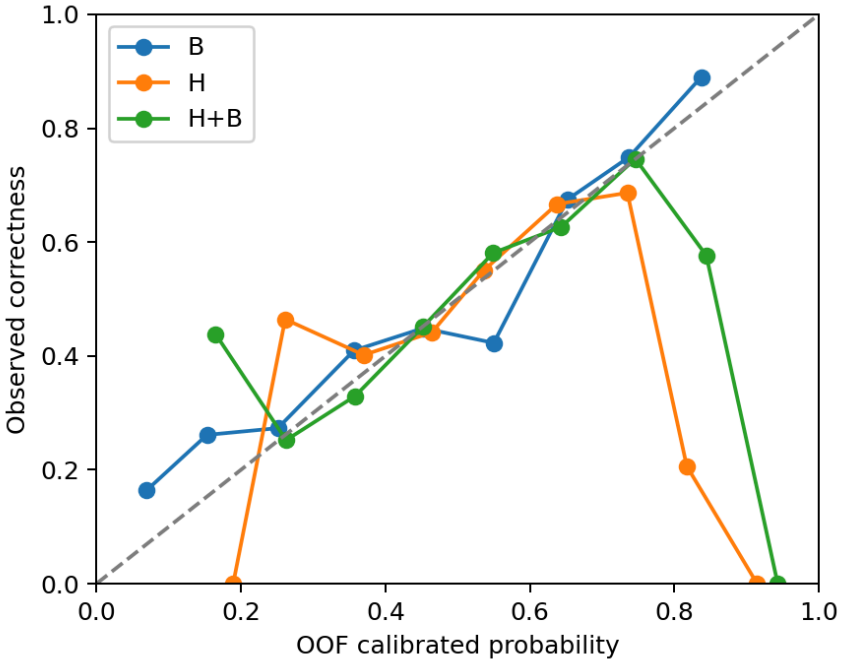


Figura 5: Probabilidade calibrada e frequência observada no teste externo.

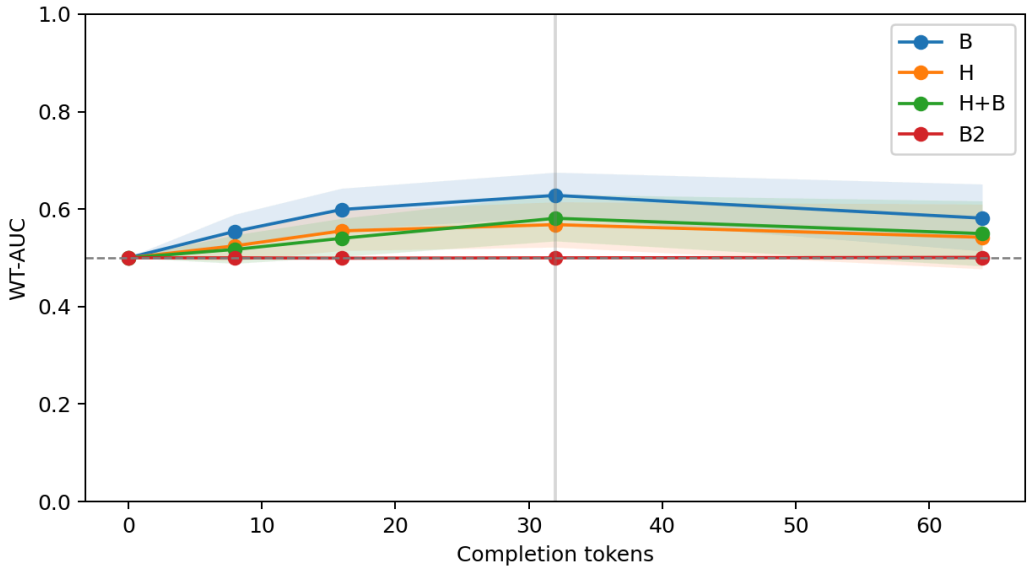


Figura 6: Discriminação por checkpoint. O checkpoint de 32 tokens é primário; demais pontos são secundários.

5.3 Replay pareado e validação em execução real

Os critérios conjuntos para afirmar benefício operacional não foram satisfeitos. O critério exige não inferioridade de resolução com margem de três pontos percentuais e economia de tokens, ambos contra verificador e B, além de suporte a Q2 e validação online com aborto efetivo. Uma falha em qualquer condição impede afirmar benefício do sistema.

N	Política	Resolução	Tokens/tarefa	Aborto falso ativo
1	B	0,462	68,7	0,033
1	H+B	0,470	70,2	0,019
1	verifier	0,477	72,2	0,000
5	B	0,659	190,0	0,146
5	H+B	0,674	192,0	0,096
5	verifier	0,697	199,9	0,000
10	B	0,750	296,0	0,197
10	H+B	0,773	300,2	0,103
10	verifier	0,795	310,5	0,000
20	B	0,811	447,2	0,297
20	H+B	0,833	460,2	0,120
20	verifier	0,848	467,3	0,000

Tabela 2: Contabilidade de continuação: tentativas abortadas consomem orçamento; parada no primeiro sucesso verificado.

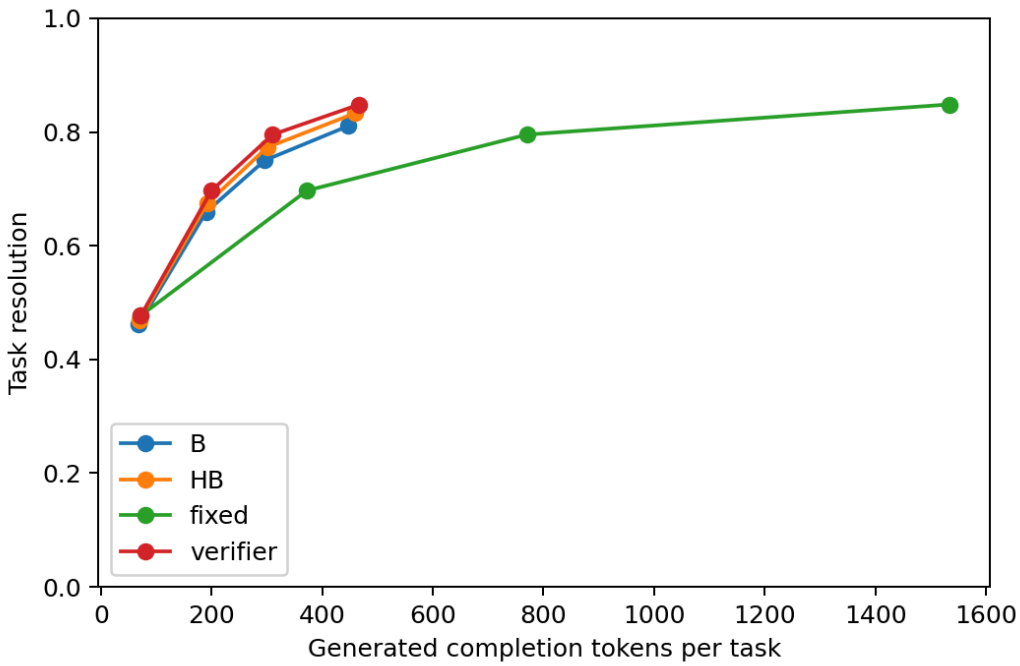


Figura 7: Custo e resolução nos mesmos fluxos de candidatos. Tokens não equivalem a latência.

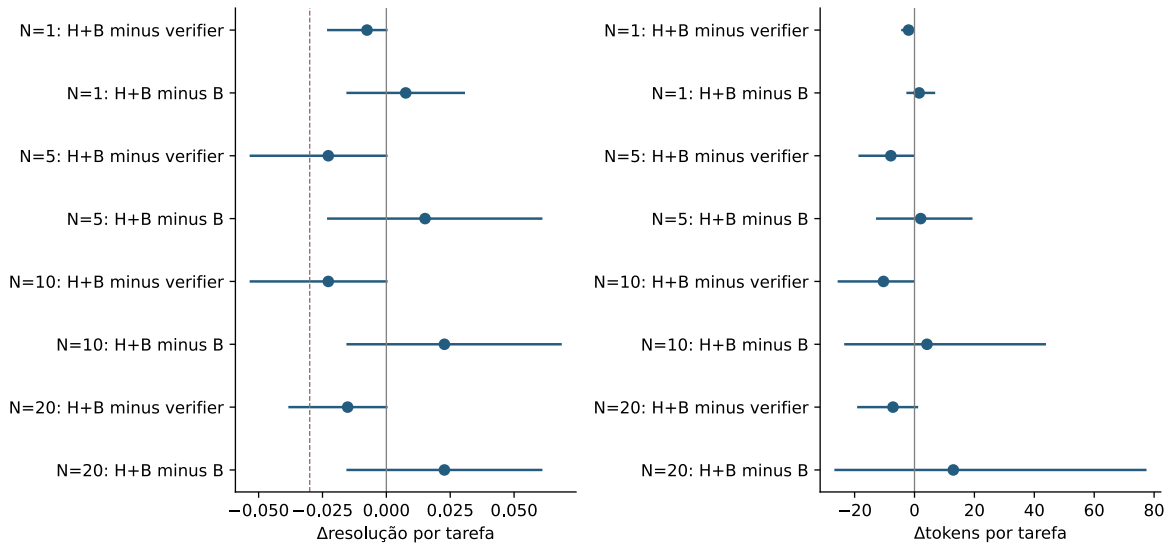


Figura 8: Diferenças pareadas H+B menos comparador por orçamento, com IC 95%. Linha tracejada: margem de resolução −0,03.

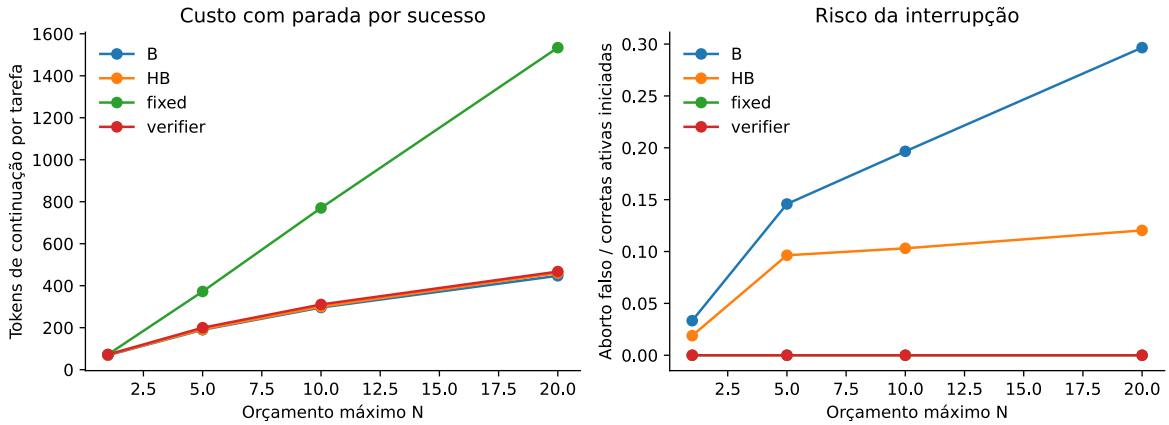


Figura 9: Custo e descarte de tentativas corretas ativas iniciadas. O denominador exclui tentativas nunca iniciadas.

A validação online incluiu 12 tarefas escolhidas por ordem lexicográfica, sem seleção por resultado. A reprodução de tokens, identidades e decisões passou. Houve aborto efetivo. Os tempos são descritivos, sem repetições suficientes para inferir economia de latência.

5.4 Transferência congelada para MBPP+

O sistema foi ajustado exclusivamente no banco principal HumanEval+ e aplicado sem reajuste, recalibração ou escolha de limiar nos rótulos alvo. O resultado é secundário e offline.

Checkpoint	Preditor	AUC	WT-AUC	Log loss
8	B	0,470	0,545	0,718
8	H	0,372	0,502	0,880
8	H+B	0,364	0,494	0,883
16	B	0,725	0,715	0,659
16	H	0,610	0,538	0,673
16	H+B	0,644	0,618	0,663
32	B	0,756	0,642	0,648
32	H	0,706	0,626	0,658
32	H+B	0,733	0,643	0,640
64	B	0,915	0,847	0,590
64	H	0,614	0,529	0,689
64	H+B	0,741	0,542	0,644

Tabela 3: Desempenho externo de transferência. Uma população homogênea não identifica WT-AUC.

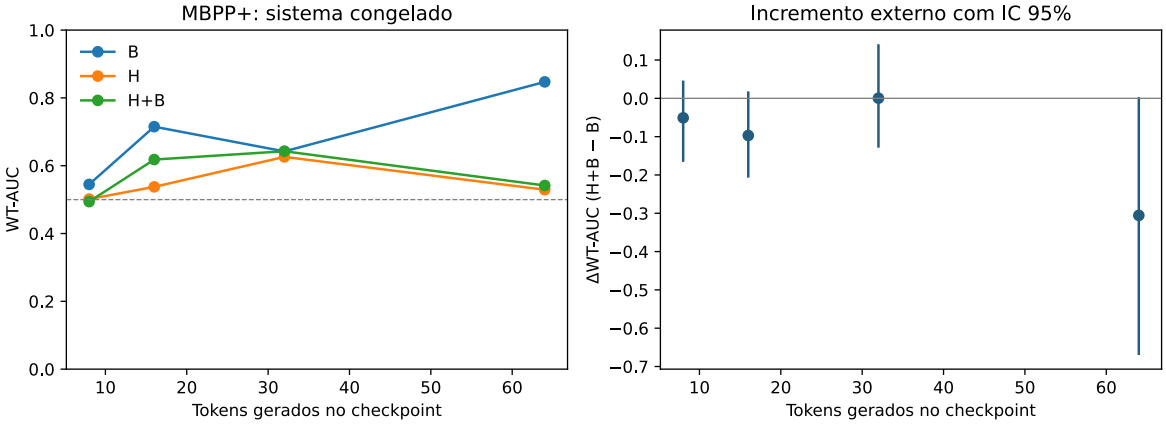


Figura 10: Transferência externa sem ajuste nos rótulos alvo. WT-AUC e contraste pareado; checkpoints são secundários.

6 Discussão e limitações

6.1 Interpretação dos resultados do desenho fixo

O incremento primário não satisfaz os critérios estabelecidos antes do banco principal. A comparação por tarefa evita confundir facilidade entre problemas com ordenação de trajetórias de um mesmo problema. O valor observado é −0,047, IC 95% [−0,099; 0,006]. Mesmo um efeito positivo não demonstra mecanismo causal: o estado é uma representação do prefixo produzido pelo próprio modelo.

Os critérios conjuntos para afirmar benefício operacional não foram satisfeitos. A conclusão operacional depende das duas comparações pareadas, do risco de perder sucessos e da execução real. O custo em tokens não mede automaticamente tempo, energia ou custo de prefill. A transferência constitui um teste de transportabilidade limitado a outro benchmark de código.

Os intervalos de bootstrap mantêm os modelos OOF fixos; não incluem toda a variabilidade do procedimento de treino. As tarefas HumanEval não são inteiramente inéditas ao histórico da pesquisa: 26 tarefas principais estão presentes em artefatos anteriores. O protocolo é de desenho fixo com execução autorizada pelo estudante; a aprovação do orientador permanece pendente e não se reivindica pré-registro independente. Resultados secundários e históricos não alteram a regra principal nem transformam um resultado nulo em confirmação.

6.2 Significado dos padrões encontrados

A discussão segue os resultados medidos para Q1–Q3, distinguindo associação, ganho incremental e utilidade operacional. Uma melhora da AUC agregada que não se repete dentro das tarefas é compatível com reconhecimento de dificuldade. Uma melhora dentro da tarefa que não supera B indica que observáveis simples podem carregar a informação operacional relevante. Um ganho de H+B sem economia mostra que decodabilidade e utilidade de interrupção não são equivalentes.

Mesmo quando as três etapas forem favoráveis, a conclusão permanece condicionada ao modelo, ao regime de amostragem, à classe de probes, aos benchmarks e ao acesso a verificação funcional. A formulação “o modelo sabe que vai errar” deve ser substituída por uma descrição probabilística da evidência medida.

6.3 Validade interna

Riscos principais incluem vazamento por tarefas repetidas, ajuste do processamento após inspeção do teste, uso de tokens futuros, descompasso entre geração e extração, escolhas múltiplas e calibração otimista. O desenho agrupado, o checkpoint confirmatório e os testes causais de prefixo reduzem esses riscos, mas precisam estar implementados e auditados, não apenas declarados.

O sinal pode refletir estilo, formatação, prefixos degenerados ou sintomas de erro sintático. Isso ainda pode ter utilidade para prever corretude funcional, mas não comprova compreensão semântica profunda do programa. Análises por classe de erro ajudam a delimitar a interpretação, desde que permaneçam secundárias.

6.4 Validade de mensuração e do verificador

Passar em uma suíte finita não equivale a satisfazer a especificação em todo domínio de entradas. Aumento de cobertura de testes reduz algumas falhas de mensuração, sem eliminá-las. Timeouts dependem de hardware e orçamento, e o parser pode afetar o rótulo. A qualidade da variável resposta precisa ser discutida junto da qualidade do probe.

O acesso a testes completos para parar a busca favorece o cenário com verificador. A comparação mede eficiência condicional a esse acesso; não estabelece o mesmo ganho em cenários sem um oráculo funcional confiável.

6.5 Validade externa e contaminação

Um único modelo pequeno e dois benchmarks de funções curtas não representam a variedade de repositórios, dependências, linguagens e tarefas de engenharia de software. Repetição de HumanEval durante desenvolvimento pode comprometer a novidade das tarefas para o pesquisador; presença de exemplos no pré-treinamento compromete inferências sobre problemas inéditos. Nenhum split de probes remove contaminação no pré-treinamento.

A mudança de benchmark pode alterar a prevalência, os prefixos e a confiabilidade do score. Isso afeta tanto AUC quanto calibração e retenção. Uma política de alta retenção precisa ser reavaliada quando o regime muda, sem atribuir garantias por analogia.

6.6 Precisão estatística e escopo computacional

A amostra relevante para generalização contém tarefas, não milhares de observações independentes. Poucas tarefas mistas e poucos sucessos em certas posições tornam intervalos largos e limiares instáveis. Mais tentativas por tarefa estimam melhor sua variabilidade, mas não substituem automaticamente mais tarefas distintas.

Probes lineares podem falhar mesmo quando existe informação acessível a modelos não lineares. Portanto, um resultado negativo delimita a classe de leitura avaliada. Expansões para classificadores mais complexos precisam de orçamento e validação próprios; não são necessárias para produzir um resultado conclusivo sobre o desenho delimitado.

6.7 Trabalhos futuros

Extensões naturais incluem avaliação com benchmarks menos sujeitos a contaminação, verificadores incompletos separados dos testes finais, reinvestimento da economia em tentativas adicionais e repetição em outros modelos. Uma comparação com representações textuais mais fortes poderia aprofundar a interpretação de ganho sobre observáveis simples.

Activation Steering constitui uma pergunta posterior: se uma direção preditiva também permite modificar a distribuição de sucesso. Essa pergunta exigiria intervenções, controles e desenho causal próprios. Não se infere a resposta a partir da existência de um probe preciso.

7 Considerações finais

A avaliação principal foi executada com 2.640 tentativas em 132 tarefas HumanEval+, precedida por piloto separado de 960 trajetórias. A validação aninhada agrupa tarefas e compara representações internas com observáveis disponíveis no mesmo prefixo. Em 32 tokens, ΔWT-AUC(H+B − B) foi −0,047, IC 95% [−0,099; 0,006] e Δlog loss foi 0,038, IC 95% [0,007; 0,071]. O resultado não satisfaz o critério de ganho incremental.

Os critérios conjuntos para afirmar benefício operacional não foram satisfeitos. A comparação com verificador e observáveis, o replay pareado e a validação real impedem atribuir ao probe economias disponíveis apenas pela parada após sucesso. A transferência para MBPP+ utiliza modelos congelados. As conclusões estão limitadas ao modelo, aos benchmarks e à população ativa avaliados. Exposição histórica, incerteza condicional e ausência de inferência robusta sobre latência são explicitadas. Replicações com novos modelos, benchmarks e tarefas externas independentes são extensões naturais.

Referências

Azaria, A., & Mitchell, T. (2023,). *The Internal State of an LLM Knows When It's Lying*. https://arxiv.org/abs/2304.13734

Brown, B., Juravsky, J., Ehrlich, R., Clark, R., Le, Q. V., Ré, C., & Mirhoseini, A. (2024,). *Large Language Monkeys: Scaling Inference Compute with Repeated Sampling*. https://arxiv.org/abs/2407.21787

Cawley, G. C., & Talbot, N. L. C. (2010). On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. *Journal of Machine Learning Research*, *11*, 2079–2107. https://jmlr.org/papers/v11/cawley10a.html

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. de O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., … Zaremba, W. (2021,). *Evaluating Large Language Models Trained on Code*. https://arxiv.org/abs/2107.03374

Di Cicco, C. (2026,). *Code Correctness Is Linearly Decodable from LLM Hidden States Before Generation*. https://arxiv.org/abs/2606.14530v3

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. *Proceedings of the 34th International Conference on Machine Learning*, *70*, 1321–1330. https://proceedings.mlr.press/v70/guo17a.html

Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., Dang, K., Fan, Y., Zhang, Y., Yang, A., Men, R., Huang, F., Zheng, B., Miao, Y., Quan, S., … Lin, J. (2024,). *Qwen2.5-Coder Technical Report*. https://arxiv.org/abs/2409.12186

Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Hatfield-Dodds, Z., DasSarma, N., Tran-Johnson, E., Johnston, S., El-Showk, S., Jones, A., Elhage, N., Hume, T., Chen, A., Bai, Y., Bowman, S., Fort, S., … Kaplan, J. (2022,). *Language Models (Mostly) Know What They Know*. https://arxiv.org/abs/2207.05221

Klassert, T. (2026,). *Introspective Uncertainty Estimation for LLM-Based Code Generation*. https://arxiv.org/abs/2609.13975v1

Liu, J., Xia, C. S., Wang, Y., & Zhang, L. (2023,). *Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation*. https://arxiv.org/abs/2305.01210

Ojala, M., & Garriga, G. C. (2010). Permutation Tests for Studying Classifier Performance. *Journal of Machine Learning Research*, *11*, 1833–1863. https://jmlr.org/papers/v11/ojala10a.html

Ruan, K., Huang, Z., Zhou, Z., Wei, Q., Lin, J., Wang, X., & Sun, H. (2026,). *Doomed from the Start: Early Abort of LLM Agent Episodes via a Recall-Controlled Probe Cascade*. https://arxiv.org/abs/2607.06503

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017,). *Attention Is All You Need*. https://arxiv.org/abs/1706.03762

Wang, Y., Zhang, P., Huang, S., Yang, B., Zhang, Z., Huang, F., & Wang, R. (2025,). *Sampling-Efficient Test-Time Scaling: Self-Estimating the Best-of-N Sampling in Early Decoding*. https://arxiv.org/abs/2503.01422

Zhu, Y., Liu, D., Lin, Z., Tong, W., Zhong, S., & Shao, J. (2025,). *The LLM Already Knows: Estimating LLM-Perceived Question Difficulty via Hidden Representations*. https://arxiv.org/abs/2509.12886

Zou, A., Phan, L., Chen, S., Campbell, J., Guo, P., Ren, R., Pan, A., Yin, X., Mazeika, M., Dombrowski, A.-K., Goel, S., Li, N., Byun, M. J., Wang, Z., Mallen, A., Basart, S., Koyejo, S., Song, D., Fredrikson, M., … Hendrycks, D. (2023,). *Representation Engineering: A Top-Down Approach to AI Transparency*. https://arxiv.org/abs/2310.01405

8 Reprodutibilidade e histórico metodológico

8.1 Artefatos e comandos

O protocolo vigente está em `RESEARCH_PROTOCOL.md`; o anterior foi preservado em `docs/protocols/RESEARCH_PROTOCOL_v0.1.md`. O texto originalmente enviado foi preservado em `tcc/archive/`. O notebook executado é `notebooks/final_research_summary.ipynb`; sua exportação HTML permite leitura sem iniciar um kernel. O manifesto `reports/final_review/evidence.json` contém os hashes dos arquivos de entrada e o escopo de cada resultado.

```
.venv/Scripts/python.exe -m pytest -q
.venv/Scripts/python.exe scripts/build_research_delivery.py --reanalyze-legacy
.venv/Scripts/python.exe scripts/build_final_notebook.py --execute
.venv/Scripts/python.exe scripts/build_manuscript.py --compile
```

A geração de evidência precisa dos arquivos históricos em `runs/`; o notebook apresenta os resultados já exportados. Os capítulos de fundamentação e método são editáveis. As seções empíricas são geradas a partir de um único manifesto, reduzindo divergência entre números no texto, tabelas e notebook.

8.2 Correções da versão 0.2

Dimensão	Implementação revisada
Desfecho	Aprovação funcional; truncamento registrado separadamente
Temporalidade	Término antes do gate; conjunto ativo estrito
Margem de logits	Após temperatura e antes do top-p
Seleção e calibração	Cross-fitting da seleção; avaliação externa por tarefa
Inferência	Associação por fold; incremento por deltas externos pareados
Política	Busca por custo em todos os limiares admissíveis, incluindo ausência de aborto
Transferência	Bundle treinado na origem, congelado antes da coleta alvo

Nenhuma alteração é apresentada como anterior a resultados já conhecidos. Há arquivos históricos de 30 tarefas HumanEval, incluindo 26 do conjunto hoje reservado para a avaliação principal. Essa exposição é diferente de vazamento entre os folds de treinamento, mas limita alegações de um teste totalmente intocado pelo pesquisador. A aprovação científica e o congelamento continuam sendo decisões reais do aluno e do orientador.

Em 14 de outubro de 2026, o estudante autorizou a execução integral. A emenda administrativa e o snapshot verificável dos fontes registram essa autorização antes do banco principal. A aprovação do orientador permanece pendente; não se infere aprovação a partir da execução e não se reivindica pré-registro independente. O arquivo normativo original foi preservado para manter a proveniência do piloto, então em andamento. A validação online usa as primeiras 12 tarefas por ID lexicográfico, sem escolha por economia observada.

8.3 Integridade e assistência computacional

Em 15 de outubro de 2026, uma sessão de coleta foi interrompida por bloqueio temporário do Windows durante a substituição atômica do manifesto JSON. A auditoria confirmou 2.433 registros completos, que foram preservados na retomada. O launcher `scripts/run_research_safely.py` acrescenta tentativas limitadas a dez segundos para essa operação de filesystem. Os fontes científicos congelados, os bytes publicados, as sementes e os procedimentos numéricos permanecem preservados. O registro administrativo, os hashes da camada de compatibilidade e o teste com bloqueio real estão documentados em `docs/windows_execution_recovery.md` e nos artefatos de recuperação em `reports/research/`.

O manifesto preservado da sessão interrompida registra pico reservado de 5.582 MiB, superior ao pico de aproximadamente 4,49 GiB do piloto. O valor da última sessão concluída, isoladamente, não resume o pico de toda a coleta.

Os testes verificam rótulos, sementes, posições, término exatamente no checkpoint, contabilidade do replay, isolamento de tarefas e ausência de ajuste com rótulos do benchmark de transferência. Nenhum teste demonstra, por si só, ausência de adaptação humana ao conjunto de dados. O histórico de decisões complementa as verificações automatizadas.

Esta versão foi preparada com assistência de um agente de programação para implementação, auditoria, organização dos artefatos e revisão textual. A autoria científica requer que o pesquisador confira a argumentação, as referências e as decisões experimentais e assuma responsabilidade pela versão submetida.